

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: Ch.Gouri Sai Sree

Roll No.: A24126552076

Date: 25-08-2026

1. TITLE

Write a JavaScript program using arrays and functions to implement a To-Do List.

2. AIM

To write and execute a JavaScript program using arrays and functions to create a simple To-Do List application that allows users to add and delete tasks dynamically.

3. OBJECTIVES

- To understand the basic concepts of JavaScript arrays and functions.
- To create and manipulate an array to store multiple tasks.
- To implement functions for adding, displaying, and deleting tasks.
- To use array methods such as push() and splice().
- To use conditional statements for input validation.
- To access and modify HTML elements using DOM methods.
- To dynamically create HTML list elements using JavaScript.
- To handle user actions using buttons and event handlers.

4. THEORY

JavaScript is a scripting language used to add logic, functionality, and interactivity to web pages. In this experiment, JavaScript is used to develop a simple To-Do List application.

The application allows the user to enter a task, add it to a list, and delete tasks when required. The tasks are stored in a JavaScript array and displayed dynamically on the webpage.

The main JavaScript concepts used in this program are:

- Array: An array is used to store multiple tasks. The program initializes an empty tasks array.
- Functions: Functions are used to perform specific operations such as adding, displaying, and deleting tasks.
- push() method: The push() method adds a new task to the end of the tasks array.
- splice() method: The splice() method removes a task from the array using its index.
- Conditional statement: An if statement checks whether the user has entered a task. If the input is empty, an alert message is displayed.
- DOM: The Document Object Model allows JavaScript to access and modify HTML elements.
- getElementById(): This method is used to access the input field and task list using their IDs.
- createElement(): This method creates a new element dynamically.
- appendChild(): This method adds the newly created list item to the task list.
- innerHTML: It is used to clear the existing task list before displaying the updated list.
- value: It is used to obtain the task entered by the user and to clear the input field after adding a task.

The basic flow of the program is:

User Input → Validation → Add Task to Array → Display Tasks → Delete Task → Update Array and Display

5. ALGORITHM / PROCEDURE

1. Create an HTML page containing a text input field, an Add button, and an unordered list to display tasks.
2. Apply CSS styles to design the To-Do List interface.
3. Initialize an empty array named tasks to store the tasks.
4. Define the addTask() function.
5. Get the task entered by the user using document.getElementById("taskInput").value.
6. Check whether the input is empty.
7. If the input is empty, display an alert asking the user to enter a task and stop the function.
8. If the input is not empty, add the task to the tasks array using the push() method.
9. Clear the input field.
10. Call the displayTasks() function to display the updated list.
11. In displayTasks(), access the taskList element and clear its existing contents.

12. Use a for loop to traverse the tasks array.
13. Create a new element for every task using document.createElement().
14. Add the task and a Delete button to the list item.
15. Add the list item to the task list using appendChild().
16. Define the deleteTask(index) function.
17. Remove the selected task from the array using splice(index, 1).
18. Call displayTasks() again to refresh the displayed list.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      text-align: center;
      padding: 50px;
    }

    .todo-container {
      background-color: white;
      width: 400px;
      margin: auto;
      padding: 25px;
      border-radius: 10px;
      box-shadow: 0 4px 10px #aaa;
    }

    input {
      width: 65%;
      padding: 10px;
      border: 1px solid #aaa;
      border-radius: 5px;
    }

    button {
      padding: 10px 15px;
      background-color: #00897b;
      color: white;
      border: none;
      border-radius: 5px;
      cursor: pointer;
    }

    button:hover {
      background-color: #00695c;
    }

    li {
```

```

        text-align: left;
        margin: 10px 0;
        padding: 10px;
        background-color: #e8f5f3;
        border-radius: 5px;
    }

    .delete {
        float: right;
        background-color: #e53935;
        padding: 5px 8px;
    }
</style>
</head>
<body>
    <div class="todo-container">
        <h1>To-Do List</h1>
        <input type="text" id="taskInput" placeholder="Enter a task">
        <button onclick="addTask()">Add</button>
        <ul id="taskList"></ul>
    </div>

    <script>
        let tasks = [];

        function addTask() {
            let task = document.getElementById("taskInput").value;

            if (task === "") {
                alert("Please enter a task!");
                return;
            }

            tasks.push(task);
            document.getElementById("taskInput").value = "";
            displayTasks();
        }

        function displayTasks() {
            let list = document.getElementById("taskList");
            list.innerHTML = "";

            for (let i = 0; i < tasks.length; i++) {
                let li = document.createElement("li");
                li.innerHTML = tasks[i] +
                    " <button class='delete' onclick='deleteTask(" +
                    i + ")'>Delete</button>";
                list.appendChild(li);
            }
        }

        function deleteTask(index) {
            tasks.splice(index, 1);
            displayTasks();
        }
    </script>
</body>
</html>

```

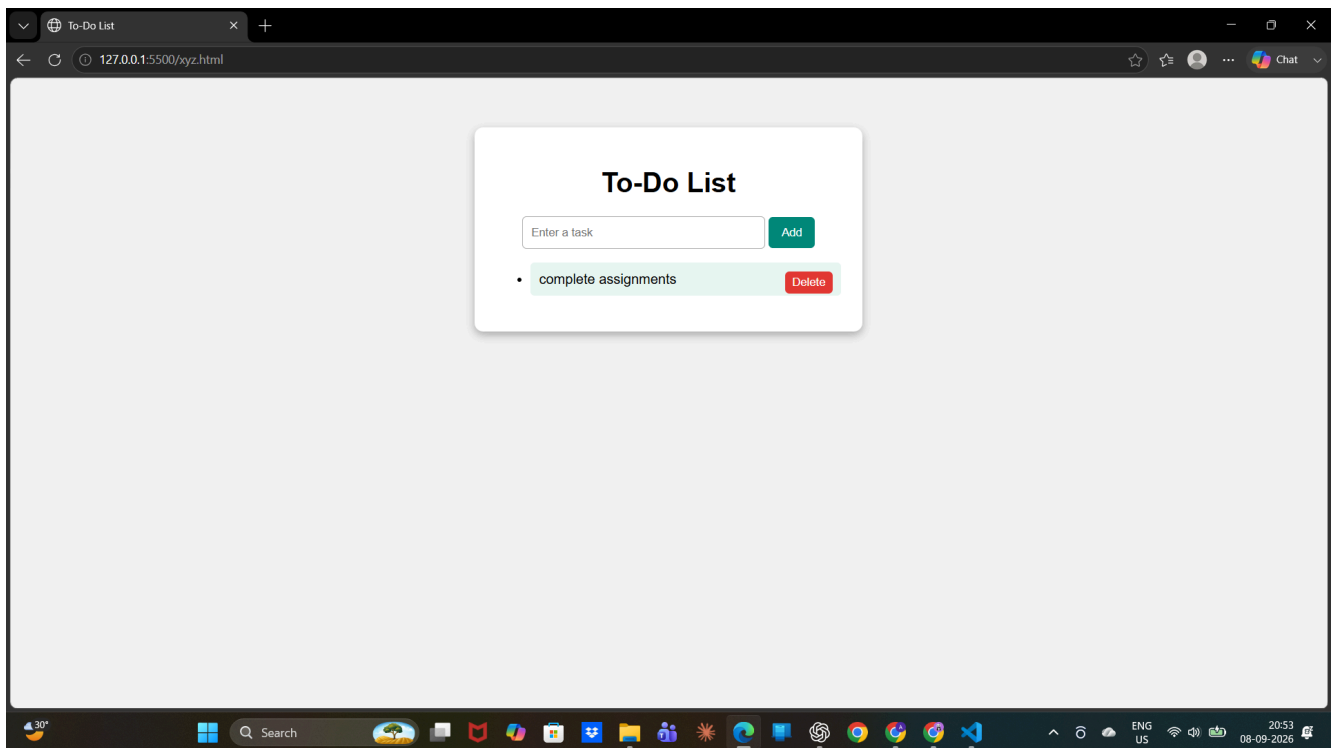
7. CODE EXPLANATION

Component	Description
let tasks = []	Creates an empty array to store the tasks.
addTask()	Gets the task from the input field, validates it, and adds it to the array.
document.getElementById()	Accesses HTML elements using their IDs.
value	Gets the value entered in the input field.
if (task === "")	Checks whether the user entered an empty task.
alert()	Displays a message when the user tries to add an empty task.
tasks.push(task)	Adds a new task to the end of the array.
displayTasks()	Displays all tasks stored in the array.
innerHTML = ""	Clears the existing task list before displaying the updated list.
for loop	Iterates through all tasks in the array.
createElement("li")	Dynamically creates a list item.
appendChild()	Adds the created list item to the task list.
deleteTask(index)	Deletes a selected task using its array index.
tasks.splice(index, 1)	Removes one task from the array at the specified index.

8. TEST CASES AND EXECUTION DATA

Test Case	Scenario	Input / Action	Expected Result	Actual Result	Status
TC-01	Open webpage	Open the HTML file in a browser	To-Do List webpage should load successfully	Webpage loaded successfully	Pass
TC-02	Add a single task	Enter Complete assignment and click Add	Task should be added to the task list	Task displayed successfully	Pass
TC-03	Add multiple tasks	Enter three different tasks and click Add after each	All tasks should be displayed in the list	All tasks displayed successfully	Pass
TC-04	Empty input validation	Click Add without entering a task	Alert should ask the user to enter a task	Alert displayed correctly	Pass
TC-05	Clear input after adding	Enter a task and click Add	Input field should become empty	Input field cleared successfully	Pass
TC-06	Delete a task	Add a task and click its Delete button	Selected task should be removed from the list	Task deleted successfully	Pass
TC-07	Delete one task from multiple tasks	Add three tasks and delete the second task	Only the selected task should be removed	Selected task removed successfully	Pass
TC-08	Delete all tasks	Add multiple tasks and delete them one by one	Task list should become empty	All tasks deleted successfully	Pass
TC-09	Add task after deletion	Add a task, delete it, and add another task	New task should be added and displayed correctly	New task displayed successfully	Pass

9. OUTPUT



10. RESULT

Thus, a JavaScript-based To-Do List application was successfully developed using arrays and functions. The program allows users to add and delete tasks dynamically. JavaScript array methods such as `push()` and `splice()`, along with DOM methods such as `getElementById()`, `createElement()`, and `appendChild()`, were successfully implemented. The program was tested using different test cases and produced the expected results.